Waking a Harness-Invoked Agent: A Blocking Receive in a Child Process and the Token Cost of Polling Instead

Vasili Gavrilov*

November 2026

Abstract

A language-model agent run by a command-line harness exists only while the harness invokes it: between turns there is no process, no socket and no event loop to deliver a message to. Coordinating several such agents on one machine is usually handed to a queue, a broker or an inter-agent protocol, which deliver to a running consumer. The missing primitive is a wakeup, and we give one: a blocking receive in a child process. The agent parks a small C program on a shared log; the program sleeps on an `inotify` watch, costs nothing while it waits, and exits when a message for its seat arrives, which is the one signal a harness already turns into an invocation. The alternative, the model polling the log itself, costs one activation per poll, and every activation is billed for the whole context, cached or not. We measure that on two models: billed tokens per poll equal the context at every interval, almost all of them cache reads, so an idle seat's bill is polls per hour times context, from cents to tens of dollars an hour for an empty board, while the parked child makes no model call at all. The transport under the wakeup is an append-only log in one directory, locked for writers, with a cursor per member, claims as exclusive file creation and presence as held locks, in under a thousand lines of C99 with no heap allocation. A send and the wake it causes take about two milliseconds whatever the log's size, since a reader starts from its own cursor. A pilot put three agent seats on one such log to repair a SWE-bench Verified issue; the official harness scored the joint patch resolved, and the whole exchange is one readable file.

1 Introduction: the misidentified constraint

Several LLM agents running on one machine, each competent in isolation, are surprisingly hard to make work together. In this paper a *harness* is the program that invokes the model and runs its tool calls (Claude Code in the measurements below); a *seat* is one agent process on the board; a *room* is a directory holding one shared log; a *frame* is one message on that log. The reflex, borrowed from distributed systems, is to reach for messaging infrastructure: a message queue, an event bus, or one of the inter-agent protocols now consolidating in the field [29, 27]. That reflex imports a service to run, identities to provision and a network to secure, and it solves the wrong problem.

Bytes move between processes fine; that was solved decades ago. The hard part is *waking a reader that, between turns, is not running at all.* An LLM agent advances only when its harness invokes it with input. In the interval there is no thread, no socket, and no event loop into which a message could be delivered. Messaging infrastructure buffers and distributes for consumers that are always live; it has nothing to say about a consumer that does not exist between turns.

This paper makes four claims. *First* (§2-§3), an LLM agent is the wrong shape for a queue: it is slow, serial, and dormant, so throughput, the thing a broker manages, is never the bottleneck.

*Independent researcher.

Second (§4-§5), the primitive an agent needs is a wakeup, and the only wakeup available to a stateless agent is the return of a blocking receive run in a cheap child process; a token-cost model, with one measured hour, shows what the poll costs when it lives in the model and why it must live in C. *Third* (§6-§7), once a wakeup is the requirement, the transport reduces to an append-only log plus advisory locking, the commit-log abstraction beneath a log-based broker without the service, realised here as `iac`, a dependency-free C99 reference implementation whose append, wake and catch-up times are measured in §5. *Fourth* (§10), a pilot: three agent seats repair one SWE-bench Verified issue over the log, and the harness scores the joint patch resolved.

The design is reduced to its essential primitive, and the artefact kept small enough to audit by reading.

2 The consumer an agent actually is

A message queue exists to serve a particular kind of consumer: one that is fast, numerous, and concurrent. An LLM agent is none of these.

Slow. A turn (read input, reason, act) costs seconds of wall-clock time and a model inference, not the microseconds of a function call.

Serial. An agent processes one message at a time. It has one context window, one attention, one output stream, and cannot parallelise its own reasoning. It is, in the strict sense, a single-threaded reader.

Dormant between turns. This is the decisive property. Between invocations an LLM agent is not a running process: no thread, no file descriptor, no socket, no event loop. Nothing can interrupt it or deliver to it. It comes into existence only when its harness re-invokes it, runs one turn, and ceases.

Locality-bounded. In practice the fleet does not grow into a throughput problem either. A single human operator steers only a handful of agents at once (a few in ordinary use, and rarely more than about ten when the work is interdependent and each agent needs review), and on one host each agent running a heavy build or toolchain consumes gigabytes of memory, which tightens the bound further. A fleet of that size, driven at human pace, never generates the message rate a broker exists to absorb. This is the author's observation from running such fleets, with no usage study behind it, and it is not a hard limit: when agents run largely autonomously, with the operator only occasionally in the loop, the operator-span bound relaxes and larger fleets become possible. The per-agent seriality above is the load-bearing point, and it does not relax: each agent is still a slow serial reader.

So on both axes, the single agent and, in ordinary use, the whole fleet, throughput is a non-problem. The queue is optimised for exactly the pressure that never arrives.

3 Why a broker is the wrong tool

A broker's job is throughput and concurrency: buffer bursty producers, fan work out to many consumers, apply backpressure, drain in parallel at speed. Pointed at a slow serial reader, none of that machinery does any work; it is a turbine bolted to something that walks.

More to the point, delivery presupposes something the agent does not have. Brokers reach consumers in exactly two ways. Either the consumer runs a *poll loop*: it calls a blocking receive, the kernel wakes it when data arrives, and it fetches. Or the broker *pushes* to a connection the consumer holds open. Both require a running consumer. An Apache Kafka consumer receives

its messages because it is a live process blocked on a poll, woken by the kernel. That is cheap, since a process blocking on a socket read costs nothing, and it is never dormant.

An agent between turns is not that process. It has no poll loop, because there is no process to run one, and no held connection to push to. So to receive at all, it needs an *external* process to hold the wait on its behalf and wake it on arrival, the construction of §4. That waking process is what matters, and it is independent of the transport beneath it: one could wrap it around a Kafka consumer, a socket read, or a file, and get the same wakeup. This is the case against the broker. Wrap its consumer in the same waking process and it can deliver the wakeup; the wakeup is required *either way*, and once it is in place the broker's throughput and concurrency machinery does no work for a slow serial reader. What remains is a dependency around a need it does not remove. Making the *model* itself poll is worse still: every check is a full inference over the whole context, paying a language model to stare at an empty mailbox.

4 The wakeup primitive

What, then, can wake a stateless agent? One signal, and the environment already supplies it: *a background child has finished*, the signal a parent process receives for free. The harness used in this paper, Claude Code, re-invokes the model when a background task it started exits; the construction below rests on that behaviour; a harness without it has only the foreground shape below, one activation per timeout. For a harness-invoked agent that return is the way an inbound message reaches it while it is dormant.

The construction follows directly. Park a blocking receive in a cheap child process:

```
recv(room, me, timeout)
```

On Linux this process sleeps on an `inotify` watch [9] of the room's log (a ∼100 ms poll elsewhere), consuming nothing while it waits, one process asleep like a blocked `epoll_wait`. When a message addressed to `me` appears, `recv` returns; the child exits; the harness re-invokes the agent, which comes alive holding the message. This is the ordinary consumer poll loop of §3, *relocated* into a process the agent can afford to keep alive, because the agent cannot be that process itself. It is the textbook choice to block in `epoll_wait` rather than busy-poll a flag [9, 10], applied to a caller that happens to be a language model, for which the poll avoided costs a full inference against a server's syscall.

Two loop shapes carry it, both from the repository's `SKILL.md` [1]. In the foreground shape the harness holds the loop: the agent blocks in a plain tool call, with a timeout under the harness's cap on one call (about 600 s in Claude Code), and issues the next `recv` itself when the call returns:

```
iac recv "$ROOM" "$ME" 500
```

A timeout is also a return, so this shape pays one model activation per timeout as well as per message. In the background shape, run as a task the harness backgrounds, a shell holds the loop, re-blocks on every timeout, and the model is invoked only when a message breaks it:

```
while :; do
  m=$(iac recv "$ROOM" "$ME" 3600) && { printf '%s\n' "$m"; break; }
done
```

A background job is not subject to the foreground cap, so the timeout can be long, and a timeout costs nothing. In either shape the seat must issue the next `recv` after every turn; a seat that forgets is awake but unreachable until something else invokes it, and the background shape needs a re-launch after each delivery, by the model or by the harness's own instructions. A message that arrives while the seat is mid-turn waits on the log until the next `recv`; nothing is lost, since the cursor has not moved.

The sleeping waiter is a plain C process, so it burns no tokens while parked, and the bill scales with messages: **one model activation per message**. The polling still happens, inside the C process, where an idle wait costs nanoseconds; inside the model every check would cost a turn. The receive model is developed at length in the reference implementation's documentation in `poll`/`epoll` terms [1].

Scope. This argument is about *stateless, harness-invoked* agents: the CLI subagent that runs a turn and stops, which is the common case today. The claim that the wakeup is a returning process holds *given* a harness whose one wake signal is a background child's exit; a different harness could expose a native file-watch or a scheduled re-invocation instead. And an agent hosted as a long-lived server is a different case: it has a socket to hold open and can be pushed to like any service. But that reintroduces the running process, the network, and the identity the minimal design exists to avoid. For an agent that only exists when invoked, a returning process is the wakeup available.

5 A token-cost model

The reason the poll must live in C is economic. Consider an agent idle for a wall-clock interval T, watching for messages, with a naive poll interval p and a context of C tokens.

If the *model* performs the poll, it is invoked T/p times, and every invocation is billed for the whole context:

$$A_{\text{model}} = \frac{T}{p}, \qquad \text{Tokens}_{\text{model}} = \frac{T}{p}\,C. \tag{1}$$

What the provider charges per token depends on its prompt cache. Within the cache lifetime (five minutes at the endpoint used here) the prefix is served from the cache at a fraction of the input rate; past the lifetime it is written to the cache again at a premium; with no cache every poll pays the full input rate. The number of *activations*, T/p, is the same under all three, and it is the activations that diverge as T grows or p shrinks.

If instead the poll lives in a C process that re-blocks on each timeout without invoking the model, the idle cost is

$$A_{\text{C}} = 0, \qquad \text{Tokens}_{\text{C}} = 0 \quad \text{(while idle, for any } T\text{)}. \tag{2}$$

A background shell loop, `while :; do recv ...; done`, re-blocks in C on every timeout and invokes the model only when a real message breaks the loop. An agent can therefore sit on a board for days at zero token cost.

Under the wakeup design, model activations scale with *work*, not with *time*:

$$A_{\text{wakeup}} = \#\{\text{messages for me}\} = O(\text{messages}), \quad \text{versus} \quad A_{\text{model-poll}} = O(T/p). \tag{3}$$

A worked example, measured. We ran exactly this. A Claude Sonnet 5 agent (model id `claude-sonnet-5`, July 2026) carrying a $C = 49{,}836$-token context (a realistically loaded agent) was asked to check for a message every $p = 30\,\text{s}$ for one hour, so $T/p = 120$ activations, against the Anthropic Messages API, which returns exact token usage per call. The token counts are the measurement; the dollar figures are those counts at the model's list rates[1] (harness and recorded usage in the paper's repository [2]):

Sonnet 5, 49,836 tokens, one hour	Activations	Input tokens billed	USD	
Model polls every 30 s, cache warm	120	5,980,320	$1.33	measured
The same, no prompt cache	120	5,980,320	$11.97	derived
`iac` wakeup (`recv` blocks in C)	**0**	**0**	**$0.00**	by construction

[1] Claude Sonnet 5 list rates: $2, $0.20, $2.50 and $10 per million input, cache-read, five-minute cache-write and output tokens, from `https://platform.claude.com/docs/en/about-claude/pricing`, read 2026-11-02.

The warm hour was billed almost entirely as cache reads (5,926,081 of the 5,980,320 input tokens; the rest the initial cache write and the per-call question; the 875 output tokens sat under the harness's 8-token cap). The no-cache row is the same token count at the input rate. None of it depends on there being a message to find. `iac` makes no call while idle, by construction; its one call is for the message. For scale, the pilot of §10 cost $2.11 for three seats over 11.9 minutes, about $3.50 per seat-hour of wall clock and $8.70 per hour a seat was running: idle polling costs the same order as working. Five agents parked for eight hours bill about $49 warm and $479 without a cache, against zero for the board.

The grid, measured. The same harness was run on Claude Fable 5.1 (`claude-fable-5-1`, November 2026) over three context sizes and three intervals, 20 minutes per cell, the three cells of one context back to back so that the prefix was written once; dollar figures at that model's list rates[2] and scaled to the hour. The persona in the padded context is a library notice board: the earlier persona, a coding agent with file and build tools, was refused by this model's safety classifier on every call of our runs, and a refused call is billed for its cache write and never commits the cache.

context, tokens	billed per poll	USD per idle hour, steady			first poll
		every 10 s	every 30 s	every 120 s	cache write
12,211	12,218	$1.37	$0.46	$0.11	$0.15
60,726	60,742	$5.89	$1.96	$0.49	$0.76
181,936	181,943	$16.6	$5.55	$1.39	$2.27

Billed tokens per poll equal the measured context plus the reply of 7 to 16 tokens, at every interval; 99 per cent of them are cache reads. The idle bill is therefore polls per hour times the per-poll bill, of which the context at the read price is 80 to 98 per cent, so the levers are the interval and the context: a loaded seat of 182k tokens checking every ten seconds bills $17 an hour for an empty board after a first poll of $2.27. Two probes on the 12k context, polling at ten- and fifteen-minute gaps and refreshing each other's entry, found both five-minute gaps read and both ten-minute gaps rewritten, which is the documented five-minute lifetime. A cell polled every six minutes read its entry three times, but it ran beside the 61k chain, which held the same prefix warm, so it shows nothing about the lifetime and is excluded. Past the lifetime the lever reverses: a 61k seat polling every ten minutes pays the write price each time, about $0.76 a poll and $4.6 an hour, more than the $1.96 it pays polling every 30 seconds warm.

Why the argument should transfer. Two models of one provider were measured, one at one interval for an hour and the other over nine cells. The activation count T/p does not depend on any of them: every current API bills a request over its whole context, cached or not, so a check costs a context whatever the model, and a prompt cache changes the price of that context and leaves the count. Cache discounts differ by provider and model, so the dollar rows above are specific to the two models at the rates in the footnotes. The wakeup removes the activations outright, and that holds whatever model sits behind the harness. Nothing in this section distinguishes `iac` from any other blocking child: `inotifywait` on the log, or `tail -f` piped to `grep`, parks at the same zero cost. What `iac` adds is in §6: addressing, a total order, claims and presence.

This is why the poll must move out of the model into a process that sleeps, and why a shell driver or the harness holds the receive loop, since a parked model context cannot [1].

[2] Claude Fable 5.1 list rates: $10, $0.25, $12.50 and $50 per million input, cache-read, five-minute cache-write and output tokens, same page and date.

Delivery latency. The transport's own contribution to a round-trip was timed on one Linux host (kernel 6.8, Intel i7-1165G7, the log on ext4 over NVMe and again on tmpfs) around the reference binary at commit `1247745`, with a C driver that spawns the receiver, lets it park, then forks and executes the sender and waits for the receiver's exit, which is what a harness observes. The log was pre-filled with frames addressed to another member, 1 KB each, to 0, 1 and 100 MB, read once so that it sat in the page cache, and the reader's cursor set either at the end (parked) or at offset zero (lagging). Each parked cell is 300 sends; each lagging cell is 30, and for those the last column is the maximum. The driver and the data are in the paper's repository [2]; the no-log row is from the repository's `scripts/bench.sh` [1], run on 2026-11-03 with its output kept beside the driver's data.

Stage	median	p99, or max at $n = 30$
`send` process, fork to exit: one frame appended under `flock`; six cells, 0 to 100 MB, ext4 and tmpfs	1.6-1.7 ms	2.0-2.2 ms
`send` to `recv` return, reader parked at the end, same six cells	2.1-2.4 ms	7.7-8.3 ms
`recv` catching up from 1 MB behind, ext4 and tmpfs ($n = 30$ each)	1.1 and 2.5 ms	1.6 and 3.5 ms
`recv` catching up from 100 MB behind, about 100,000 frames ($n = 30$ each)	48-51 ms	59-60 ms
first message into a room with no log yet, poll tick ($n = 30$)	101 ms	101 ms

The sender's process spin-up dominates: the append itself and the `inotify` event are a fraction of the 1.7 ms, and the wake adds 0.4 to 0.8 ms at the median. Wake times are bimodal, 71 per cent near 2 ms and 29 per cent between 5 and 8 ms; the cause was not isolated. One wake in 1,800 took 102 ms: a missed `inotify` event caught by the 100 ms fallback poll, which bounds the cost of a miss. The log's size does not enter: a reader parked at the end of 100 MB wakes in the same 2.2 ms as at the end of an empty log, because it reads forward from its own cursor. A reader that has fallen behind pays for the frames it skips, about 2 GB/s from the page cache, so 100 MB of lag costs 50 ms; at 1 MB the two filesystems differ by 1.4 ms on a 30-run cell, which is unexplained. Only the first message into a not-yet-created room pays the 100 ms poll tick, because the watch attaches once the `log` file exists. The scale on the other side is the model turn, seconds to minutes: in the pilot of §10 three seats exchanged 14 messages in 11.9 minutes, so the transport is three to four orders below the turn.

6 The minimal transport

Once the requirement is a wakeup, the transport can be almost nothing: append a line, lock it, read forward, block on receive. Concretely, the reference design [1] is:

a ROOM	is a directory
the LOG	`<room>/log`: one append-only, totally-ordered stream
a CURSOR	`<room>/<name>.cur`: each member's read offset
the ROSTER	`<room>/roster/<name>`: presence

Every message is appended to the log *once*, whatever its audience, so a broadcast is one write for any number of readers. Each member reads the shared stream forward from its own cursor and delivers only the frames addressed to it, giving a single total order, a real conversation, with $O(1)$ broadcast and point-to-point or subset delivery for free. A frame is length-prefixed, `<from>|<to>|<epoch>|<len>` followed by the body, so a body may contain arbitrary bytes. The

recipient field carries the whole addressing scheme: `*` (broadcast to all but the sender), a name (point-to-point), a comma list (subset), or `?` (a work item claimed by exactly one free member).

Advisory file locking (`flock`) does three jobs at once. It *orders appends*: a sender takes an exclusive lock and writes with a single `writev`, so concurrent senders never interleave and the log stays a total order. The order the record vouches for is byte position; the epoch in the header is the sender's claim. A short write (disk full, a signal mid-write) leaves a torn frame at the tail, and a header whose length field is wrong desynchronises every reader from that offset: the reader stops with an error rather than resynchronising, so one malformed append halts the room for all readers until it is repaired. Nothing calls `fsync`; the log is durable to the page cache, not across a power loss. It *backs presence*: a parked `recv` or a `hold` beacon takes a shared lock on its roster entry for the life of the wait, and `who` probes the lock: held means online. And it *guards claims*: a `?` work item is won by an atomic `O_CREAT|O_EXCL` create keyed on the message's log offset, so the job is claimed by one worker at a time with no coordinator. A claim left unacknowledged past a time-to-live is presumed dead and taken by the next free worker, so a crash mid-task does not lose the work; a worker that is slow rather than dead then runs the job a second time, so delivery of a claim is at-least-once and the task must tolerate a repeat. Presence is a held lock rather than a heartbeat, so the operating system clears it the instant the holder dies, even on `SIGKILL`: nothing to reap, and no process-id-reuse guessing.

Two old lineages meet in this shape, and the shape is the paper's point. The append-and-lock-while-writing is how Unix local mail is delivered: a mailbox the delivering agent locks while it appends. Mail is per-recipient; this log is shared, totally ordered, and consumed by many readers, each from its own offset, filtering by address. That shape, an append-only commit log read forward through per-reader offsets with broadcast as a single append, is the abstraction a log-based broker is built on [4]: a single-partition Kafka topic with consumer offsets, or a shared bulletin board in the older idiom. The paper argues against *brokers* and keeps the *log*. For a slow serial consumer the log needs no service around it, only a file and a lock.

The record is auditable by construction. The log is plain text on the local filesystem, so the entire coordination history (who said what, to whom, in what order) is one append-only file: readable with `cat`, searched with `grep`, followed with `tail -f`, diffed, versioned, archived, with no service running. Brokers keep their record in binary formats readable only through their own tooling (Kafka's segment files, an AMQP store) or in a remote queue with no local artefact at all. For a system whose participants are autonomous agents, a record a human can read without special tooling is a safety property: every message is on the record, the record is legible, and it is as trustworthy as the least trustworthy participant (§9). What the agents did outside the log, in the shared checkout, is not on it. The same transparency is the cooperative-trust boundary of §9: every participant reads the whole log, so `iac` trades confidentiality between agents for auditability by the operator who runs them.

A shared log also enables peer review, which a queue cannot. A queue delivers each message to one consumer and drops it; a fan-out topic hands each subscriber its own copy; a log-based broker that retains the record keeps it in a remote store behind a round trip. On a shared local plain-text log every agent reads the same stream, re-readable at negligible cost, and that visibility is the precondition for cross-checking: in the earlier session recorded in the repository (§10) one agent's proposed fix was refuted by another on a fine point of lock semantics before it could land. This is the reason the *blackboard* architecture was valued in classical AI [5]: its knowledge sources did not message one another point to point, they posted to and read from one visible workspace, and many fallible specialists refining the same partial solution is what let the system converge and correct. A queue is the point-to-point model the blackboard was defined against; `iac` brings the blackboard's readable workspace to LLM agents at the cost of a file, so the auditability above also holds *during* the work. It turns on proximity: because the

log sits on the same host, re-reading the whole history is a local read, and cross-checking runs continuously rather than as an occasional audit.

7 Reference implementation: `iac`

`iac` is 989 lines of C99 at commit `1247745` (`wc -l` over the eight engine files, tests excluded; `grep -c malloc` finds none), with no dependencies beyond a POSIX C library: `flock`, `writev`, `pread`/`pwrite`, `dirent`, and `inotify` on Linux with a portable poll fallback. It builds with a stock compiler and ships as one binary; there is no daemon, no runtime, and no account. The command surface is eleven verbs (`send`, `recv`, `drain`, `ack`, `ask`, `join`, `leave`, `hold`, `who`, `log`, `compact`), and the documentation names three receive-loop shapes with their cost trade-offs: draining the mailbox inline while already awake, a background `recv` that re-invokes on exit, and a shell `while`-driver that idles at zero token cost [1]. Two `recv` flags push more of the bookkeeping into the sleeping child, on the same principle: `-a` delivers the whole burst already queued in one return, so a seat pays one model turn per burst rather than one per message plus a drain; `-e` returns with a distinct exit code once every other member has been absent for a set time, so the wakeup thesis extends to absence, and a seat never spends a turn discovering that its peers are gone.

The engine follows the core rules of safety-critical C. There is no heap allocation on any path, so peak footprint is a function of the fixed struct sizes: a one-kilobyte room and a one-gigabyte room run in the same memory. String operations are bounded (`snprintf` only, never `strcpy`/`sprintf`); and the main receive and scan paths, which hold an open file across several exits, use single-exit `goto` cleanup. These are the no-heap, bounded-string, and single-exit rules of the NASA/JPL *Power of Ten* [31] and MISRA-C:2012 [32] (rule 21.3 bans dynamic allocation), in the K&R style [33] as carried into modern systems practice by Robbins [34], and they are recorded as a coding standard the implementation conforms to [1]. The engine does not claim the *full* Power-of-Ten discipline, but the memory-safety rules that eliminate C's dominant defect classes hold throughout. Continuous integration builds and runs an end-to-end suite (which drives the real binary through point-to-point, broadcast, ordering, claim recovery, and presence) on Linux and macOS, with AddressSanitizer and UndefinedBehaviorSanitizer lanes on both.

8 Related work

The space around `iac` is crowded, and growing.

In-process agent frameworks (CAMEL [18], AutoGen [21], ChatDev [19], LangGraph, CrewAI, MetaGPT [22]) orchestrate several models inside a single runtime, typically Python, with function calls and shared memory. They are a different layer: coordination *within* a process, not delivery *between* independent processes.

Shared boards for coding agents are the nearest work. AgentRoom [23] gives concurrent coding agents file-level claims, an append-only broadcast log, and peer status, as MCP tools over a CRDT-merged workspace; at matched compute it reports that the coordination tools, not the shared filesystem, carry the gain, and that a homogeneous pair beats a solo agent's abandonment rate by an odds ratio of 13.7. PatchBoard [24] replaces free-form dialogue with validated patches to a shared structured state; two further systems rebuild the blackboard around LLM knowledge sources selected by board content [25, 26]. These confirm the shape this paper argues for, from a different direction, and they differ from `iac` on exactly its axis: in each, an agent learns of a peer's post by *polling* between subtasks, paying an inference per check, where an `iac` seat parks a C child on the log and pays nothing until a peer posts (§4-§5).

Same-host agent tools are the closest cousins in mechanism. `tap` is a file-based protocol for heterogeneous agent collaboration, with markdown messages that persist across restarts,

reported with a multi-week self-application [27]; `OpenClaw` runs agents behind an always-on Node.js gateway that brokers their messages and pushes events to connected clients [28]; and some harnesses now add inter-agent messaging, a shared task list, and file locking natively. Where a harness has native messaging, the case for a separate binary is the pair the native channel cannot join, a seat in one harness and a seat in another on one log, a record the operator can `tail -f`, and cursors that survive a harness restart. `tap` and `OpenClaw` each carry a runtime (a protocol layer, Node and npm); `iac` has none.

Inter-agent protocols form the enterprise layer above all of this: A2A, introduced by Google in 2025, since donated to the Linux Foundation and reaching a v1.0 specification in 2026, positioned as an agent-to-agent standard across the major clouds [29]; MCP, the de-facto agent-to-tool standard, donated by Anthropic to the Linux Foundation's Agentic AI Foundation [30]; and ACP and ANP. These are networked, cross-organisational, and heavy by necessity; they answer a different question than a same-host board.

Three bodies of work pose the paper's question from the other side. MemGPT [15] treats the model as an operating-system process woken by events and timers, and AIOS [16] schedules agents as an OS schedules processes; both keep a running process around the model, which is the thing a harness-invoked agent does not have. Sleep-time compute [17] proposes spending tokens while the agent is idle; §5 is the bill for idling in the model at all. The classical lineage is explicit; `iac` sits in two of its threads. The first is coordination through a shared space: the *blackboard* architecture of classical AI, in which independent knowledge sources post to and read from a common board by relevance [5]; and Gelernter's *tuple spaces* (Linda), whose `out`/`in`/`rd` primitives include a destructive one-taker `in` that is structurally `iac`'s `?` claim [6]. `iac` is, in effect, a blackboard with a total order. The second is the *commit log*: an append-only, totally-ordered record consumed through per-reader offsets [3, 4, 7], the data model beneath modern brokers, which stripped of the service is exactly `iac`'s log and cursors. Alongside these sit the ordinary IPC primitives (named pipes, Unix-domain sockets, ZeroMQ, D-Bus), the explicit event delivery of `epoll` and its ancestors [11], mbox [13] and Maildir [14] for file-backed mailboxes (Maildir's lockless delivery by `rename` is the alternative to `flock` this design did not take, because a shared total order needs one file), the actor model [8], in which an entity's whole life is a receive loop over a mailbox, and Erlang's selective receive [12], which is what `recv` does when it filters a shared stream by address. `iac` competes with none of the tools above on features or scale; its axis is minimal, dependency-free, same-host, harness-agnostic, chosen for install-simplicity, auditability, and longevity. In a field this volatile, the smallest thing that works is often the thing still working when the frameworks have moved on.

9 Limitations

The design is deliberately narrow, and the narrowness has edges.

Same host. Coordination is over a shared filesystem. Over NFS, `flock` is emulated with byte-range locks and is local to one client under some mount options, `inotify` reports only local writes so a parked reader on another host never wakes and the 100 ms poll is the only path, and `O_EXCL` is atomic from NFSv3 on; the design has no cross-host story without swapping the log for a socket while keeping the framing. On macOS there is no `inotify`; the build there is the poll fallback.

Trust. Every seat is a process of the operator's uid with write access to the whole room. Any seat can post under any name, read every message regardless of its addressing, rewrite or truncate the log, move another member's cursor, create or delete a claim file, and hold a shared lock on another member's roster entry, so presence is spoofable too. Filesystem permissions separate the room from other users of the host and leave the seats equal to each other; the room should be a directory the operator owns, mode 0700, outside any shared temporary directory, and `send` does not refuse a symlinked log. Names are validated to `[A-Za-z0-9_-]` and bounded,

so a name cannot escape the room directory or forge a header. A frame body is untrusted model input: a seat that has read hostile text relays it to the room, and every peer reads it under a colleague's name; the mitigation belongs to the harness, which should deliver a body delimited as data with the sender marked unverified. There is no quota or rate limit: a seat's token bill is set by what its peers choose to send it, and `-a` delivers a whole burst into one context. Presence means a file descriptor is held, so a stopped or hung seat reads as online. One reader process per member at a time: two `recv`s on one cursor race. A per-frame MAC with a room key would reject a writer without the key and nothing else; binding a frame to its sender needs a key per seat that the other seats cannot read, which under one uid every other seat can read, from a file or from the process, and covering truncation and reordering needs each frame to carry its offset and the previous frame's digest. The artefact does none of this; it is a board for seats the operator already trusts with the operator's own shell.

Scope of coordination. `iac` carries messages and claims on work items. It does not lock files or arbitrate a shared working tree; three seats on one checkout can still edit the same file, and keeping them apart is the operator's convention or separate worktrees.

Scale. A single append lock serialises all sends at about 1.7 ms each (§5), a ceiling near 600 messages a second for one room, and a reader that falls behind pays about 0.5 ms per megabyte of frames it skips. Both are far above coordination traffic, and the answer at high traffic is to shard into more rooms. The log grows unbounded and is reclaimed by a manual compaction pass that drops frames every reader has passed and re-keys cursors and claims; it must run with no reader parked, since a parked watch follows the old inode. Bounding growth automatically is declared future work [1].

Harness-specificity. The wakeup rests on the harness re-invoking on child exit; it is scoped to stateless agents (§4).

Obsolescence. The tool may well be subsumed by native coordination features in the agent harnesses themselves. If so, the argument, that the primitive is a wakeup and that for a serial consumer minimality is the answer, is the part intended to outlast the artefact.

10 Pilot: three seats on one log

Three coding-agent seats (Claude Sonnet 5, headless, one process each) were put on an `iac` room to fix one SWE-bench Verified [20] issue in Django (`django__django-11141`), with a shared checkout and a brief that asked each seat to post its diagnosis before reading the others'. The room log records 14 messages over 11.9 minutes: three independent diagnoses within 11 seconds, a claim race on `loader.py` settled by the log's own order, a split into fix, tests and release notes, and a seat that lost the claim turning itself into a verifier. The seats cost $2.11 in total, and two of the three ended at the harness's turn cap; the joint patch was scored *resolved* by the official SWE-bench harness. The record (brief, room log, seat usage, patch and the harness report) is `iac/experiment/pilot/` [2]. The run was a pipeline pilot, not a comparison, and it says nothing about boards against solo agents.

This is an existence proof with $n = 1$: one issue, one machine, one operator, the author's own seats. It shows that a plain-text log with a wakeup carries a three-party repair to a verified result and that the record of how is one file. It does not show that the approach generalises across teams or scales. An earlier session of the same kind, in which agent seats on the board fixed a presence bug in the C, ported it to macOS and revised the documentation, coordinating over the board itself, is recorded in the repository [1]; it is the source of the cross-check example in §6.

11 Conclusion

The instinct to solve agent coordination with a broker imports the wrong problem. For a consumer that is slow, serial, and dormant between turns, throughput was never the bottleneck; the missing primitive is a wakeup, and the only wakeup a stateless agent can receive is the return of a blocking receive run in a cheap process the harness re-invokes on exit. Grant that, and the transport reduces to an append-only log, advisory locks, and a blocking `recv`: the commit-log abstraction a log-based broker is built on, stripped of the service.

Twenty-five years ago the author's answer to the same coordination question was a running peer-to-peer broker, `ljms`.[3] In this iteration the broker was the part to delete. The surviving parts, the log, the lock and the blocking receive, are old. The participant is new, an agent that can be woken by nothing but a returning process, and the design gives it that and no more.

Disclosure

The text was drafted by the author and edited with Claude; the author takes responsibility for its contents. Claude Sonnet 5 and Claude Fable 5.1 appear in the measurements as the systems under test, Sonnet 5 in the pilot.

References

[1] V. Gavrilov, `iac`: reference implementation, tests, benchmark script and documentation (`README.md`, `doc/dev/RECEIVE_MODEL.md`, `SKILL.md`, `doc/ROADMAP.md`, `doc/CASE_STUDY_agents_on_ais.md`). https://github.com/Anode1/iac, 2026. The coding standard the engine conforms to is `doc/dev/STYLE.md` in https://github.com/Anode1/ais.

[2] V. Gavrilov, token-bill and transport benchmark harnesses with their recorded results. Deposited with this paper, https://doi.org/10.5281/zenodo.21206970; source at `iac/experiment/` in https://github.com/Anode1/articles, 2026.

[3] J. Kreps, N. Narkhede, J. Rao, *Kafka: a Distributed Messaging System for Log Processing*. NetDB, 2011.

[4] J. Kreps, *The Log: What Every Software Engineer Should Know About Real-Time Data's Unifying Abstraction*. LinkedIn Engineering, 2013-12-16.

[5] L. D. Erman, F. Hayes-Roth, V. R. Lesser, D. R. Reddy, *The Hearsay-II Speech-Understanding System*. ACM Computing Surveys 12(2), 1980. See also H. P. Nii, *Blackboard Systems*, AI Magazine 7(2-3), 1986.

[6] D. Gelernter, *Generative Communication in Linda*. ACM TOPLAS 7(1), 1985.

[7] P. O'Neil, E. Cheng, D. Gawlick, E. O'Neil, *The Log-Structured Merge-Tree (LSM-Tree)*. Acta Informatica 33, 1996.

[8] C. Hewitt, P. Bishop, R. Steiger, *A Universal Modular Actor Formalism for Artificial Intelligence*. IJCAI, 1973.

[9] M. Kerrisk, *The Linux Programming Interface*. No Starch Press, 2010. Chapters 19 (`inotify`), 55 (file locking) and 63 (`epoll`).

[3] A peer-to-peer message broker with broadcast and multicast, 2001, unpublished; noted in the lineage section of the repository's `README.md` [1].

[10] V. S. Pai, P. Druschel, W. Zwaenepoel, *Flash: An Efficient and Portable Web Server*. USENIX ATC, 1999.

[11] G. Banga, J. C. Mogul, P. Druschel, *A Scalable and Explicit Event Delivery Mechanism for UNIX*. USENIX ATC, 1999.

[12] J. Armstrong, *Making Reliable Distributed Systems in the Presence of Software Errors*. PhD thesis, KTH, 2003.

[13] E. Hall, *The application/mbox Media Type*. RFC 4155, 2005.

[14] D. J. Bernstein, *Using maildir format*. https://cr.yp.to/proto/maildir.html.

[15] C. Packer, S. Wooders, K. Lin, et al., *MemGPT: Towards LLMs as Operating Systems*. arXiv:2310.08560, 2023.

[16] K. Mei, X. Zhu, W. Xu, et al., *AIOS: LLM Agent Operating System*. arXiv:2403.16971, 2024.

[17] K. Lin, C. Packer, S. Wooders, et al., *Sleep-time Compute: Beyond Inference Scaling at Test-time*. arXiv:2504.13171, 2025.

[18] G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin, B. Ghanem, *CAMEL: Communicative Agents for "Mind" Exploration of Large Language Model Society*. NeurIPS 2023; arXiv:2303.17760.

[19] C. Qian, W. Liu, H. Liu, et al., *ChatDev: Communicative Agents for Software Development*. ACL 2024; arXiv:2307.07924.

[20] C. E. Jimenez, J. Yang, A. Wettig, et al., *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* ICLR 2024; arXiv:2310.06770. SWE-bench Verified: OpenAI, 2024-08-13.

[21] Q. Wu, G. Bansal, J. Zhang, et al., *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*. arXiv:2308.08155, 2023.

[22] S. Hong, M. Zhuge, J. Chen, et al., *MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework*. ICLR 2024; arXiv:2308.00352.

[23] S. Cho, D. Lee, *AgentRoom: Concurrent Multi-Agent Coding in a CRDT-Backed Shared Workspace*. arXiv:2608.23740, 2026.

[24] S. Zhang, Y. Shi, L. Wang, *PatchBoard: Schema-Grounded State Mutation for Reliable and Auditable LLM Multi-Agent Collaboration*. arXiv:2605.29313, 2026.

[25] B. Han, S. Zhang, *Exploring Advanced LLM Multi-Agent Systems Based on Blackboard Architecture*. arXiv:2507.01701, 2025.

[26] A. Salemi, M. Parmar, P. Goyal, Y. Song, J. Yoon, H. Zamani, T. Pfister, H. Palangi, *LLM-Based Multi-Agent Blackboard System for Information Discovery in Data Science*. arXiv:2510.01285, 2025.

[27] M. Kim, *tap: A File-Based Protocol for Heterogeneous LLM Agent Collaboration*. arXiv:2606.14445, 2026.

[28] OpenClaw Foundation, *OpenClaw*: a self-hosted multi-agent gateway (MIT). https://github.com/openclaw/openclaw, 2026.

[29] *Agent2Agent (A2A) Protocol*. Introduced by Google, 2025; Linux Foundation project; v1.0 specification, 2026. https://a2a-protocol.org.

[30] *Model Context Protocol* specification. Introduced by Anthropic, 2024; a project of the Agentic AI Foundation (Linux Foundation) since December 2025. https://modelcontextprotocol.io.

[31] G. J. Holzmann, *The Power of Ten: Rules for Developing Safety-Critical Code*. IEEE Computer 39(6), 95-99, 2006. DOI 10.1109/MC.2006.212.

[32] MISRA, *MISRA C:2012, Guidelines for the Use of the C Language in Critical Systems*. MIRA Ltd, 2013; third edition 2019. Rule 21.3.

[33] B. W. Kernighan, D. M. Ritchie, *The C Programming Language*, 2nd ed. Prentice Hall, 1988.

[34] A. Robbins, *Linux Programming by Example: The Fundamentals*. Prentice Hall, 2004.